SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Informatics and Statistics
Department of Computer Science

Advanced Architectures

Author:
Simone Lidonnici

18 march 2026

Contents

E Exercises **1**
E.1 Exercises on performance . . . 1
E.1.1 Amdahl's Law . . . 1
E.1.2 Gustafson Law . . . 2
E.1.3 Processor Performance . . . 2
E.1.4 Circuit area and time . . . 3
E.2 Exercises on number representation . . . 5
E.2.1 Multiplication of two complement number . . . 5
E.2.2 Sum with redundant number system . . . 5
E.2.3 Simmetric number representation . . . 6
E.2.4 Conversion between decimal and signed digit . . . 7

E

Exercises

E.1 Exercises on performance

E.1.1 Amdahl's Law

Amdahl's Law

The Amdahl's law gives a bound on the speedup of a program that is enhanced (parallelized) based on the fraction of program that we can enhance α. We can calculate the time with the enhancement:

$$T_e = (1-\alpha)T + \alpha\frac{T}{S_e}$$

In which T_e is the time with the enhancement, S_e the speedup of the enhanced part and T the base program time. For parallelization with p process we can assume $S_e = p$. We can calculate the speedup as:

$$S = \frac{1}{(1-\alpha) + \frac{\alpha}{S_e}}$$

Another important metric is the efficiency with p processes:

$$E = \frac{S}{p} = \frac{1-\alpha}{p} + \alpha$$

Example:
We have a processor that does:

- 20% of the time FPSQR
- 50% of time FP

We have 2 option of enhancement:

1. Speedup FPSQR by 10 times
2. Speedup FP by 1.6 times

We calculate the speedup of the two options:

$$S_1 = \frac{1}{(1-0.2) + \frac{0.2}{10}} = \frac{1}{0.82} \simeq 1.22$$

$$S_2 = \frac{1}{(1-0.5) + \frac{0.5}{1.6} = \frac{1}{0.8125} \simeq 1.23}$$

So, the second one is slightly better.

E.1.2 Gustafson Law

Gustafson Law

The Gustafson Law uses the opposite approach in respect to the Amdahl law, so using enhanced time (parallelized) T_e with p processes, we can estimate the sequantial time based on the fraction of program that we can enhance α:

$$T = (1-\alpha)T_e + \alpha p T_e$$

The speedup can be calculated:

$$S = (1-\alpha) + \alpha p$$

Example:
We have a program that can be parallelized for 50% and we have two processors, one with 2 core and the other with 4. The speedup that the two processors can reach is:

$$S_1 = (1-0.5) + 2 \cdot 0.5 = 1.5 \implies E_1 = \frac{1.5}{2} = 0.75$$

$$S_2 = (1-0.5) + 4 \cdot 0.5 = 2.5 \implies E_2 = \frac{2.5}{4} = 0.625$$

So, with 4 cores we have more speedup but less efficiency.

E.1.3 Processor Performance

Processors uses clock cycles (CC) to run at costant rate. For a program we can calculate the CPU time for a program as:

$$CPU \text{ time} = CC \text{ program}/C \text{ rate} = CC \text{ program} \cdot C \text{ time}$$

We can count the number of instruction executed (IC) and knowing the CC per program we can calculate the CC per instruction (CPI):

$$CPI = C \text{ program}/IC$$

With the previous formulas we can redefine the CPU time as:

$$CPU \text{ time} = IC \cdot CP \cdot C \text{ time}$$

It is useful to also calculate the total clock cycle of a program considering the different types of instructions:

$$CC \text{ program} = \sum_{\forall i} IC_i \cdot CPI_i$$

In which i is a type of instruction.

If on a processor we decrease the CPI of a specific instruction, we can calculate the new CPI by subtracting the weighted difference:

$$CPI_e = CPI - \%instr(CPI_{old-instr} - CPI_{new-instr})$$

In which $\%instr$ is the fraction of the instruction enhanced, $CPI_{old-instr}$ is the CPI of that specific instruction before the enhancement and $CPI_{new-instr}$ after the enhancement.
Example:
We have a processor that does:

- 2% of the time FPSQR with 20 CPI
- 25% of time FP with 4 CPI
- Other instruction with 1.33 CPI

We have 2 option of enhancement:

1. Decrease the CPI of FPSQR to 2
2. Decrease the CPI of FP to 2.5

The starting CPI average is:

$$CPI_{old} = (4 \cdot 0.25) + (1.33 \cdot 0.75) = 2$$

The enchanced CPI and speedup are:

$$CPI_1 = 2 - 0.02(20 - 2) \simeq 1.64 \implies S_1 = \frac{2}{1.64} \simeq 1.22$$

$$CPI_2 = (2.5 \cdot 0.25) + (1.33 \cdot 0.75) \simeq 1.625 \implies S_2 = \frac{2}{1.625} \simeq 1.23$$

So, the second one is slightly better.

E.1.4 Circuit area and time

To approximate the time and area of a circuit we relwte them to the area and speed of the simplest gates. Each simple gate with 2 input count as 1 gate for area and 1 gate delay unit for time. The only exception is the xor gate which counts as 2 because is implemented with 2 basic gates.
To take in account the number of inputs of gates we count a m input gate as:

- Time: $\lceil \log m \rceil$ gates
- Area: $m - 1$ gates

From now on, we will define the gate unit for area as A and the gate delay unit for time as T.
Some simple adders have this costs:

1. Half adder:

- Time: 2T
- Area: 3A

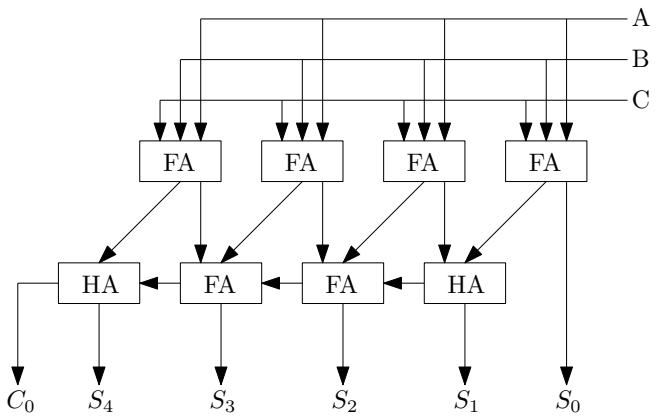
2. Full adder:

   - Time: 4T
   - Area: 7A

3. Ripple-Carry adder with n bits:

   - Time: $4n$T
   - Area: $7n$T

Example:
We have 3 operands of 4 bits and we want to sum them using the Carry-Save Adder below:



We can calculate the time and area of this adder in respect to the RCA for the same 3 4-bits operands:

1. Carry-Save Adder:

   - Time: FA + HA + 2FA + HA = 4 + 2 + 2·4 + 2 = 16T
   - Area: 6FA + 2HA = 6·7 + 2·3 = 48A

2. Ripple-Carry:

   - Time: 2RCA (4 bits) = 2·4·4 = 32T
   - Area: RCA (4bits) = 7·4 = 28A (we can reuse the same two times)

The speedup for both time and area is:

- Time: $\frac{32}{16} = 2$
- Area: $\frac{28}{48} \simeq 0.6$

E.2 Exercises on number representation

E.2.1 Multiplication of two complement number

To multiply two operands written in two's complement we can use a easier trick in respect to doing the multiplication the normal way. Let A and B be the two operands we want to multiply, for every digit a_i in A:

- If (a_i, a_{i-1}) is (0,0) or (1,1): nothing to add
- If (a_i, a_{i-1}) is (1,0) or the firts digit as 1: sub B
- If (a_i, a_{i-1}) is (0,1): add B

The add and sub are shifted based on the position of the i-th digit in A.
Example:
We want to multiply 10 and -7, so in two's complement:

$$A = 01010 \qquad B = 11001$$

The multiplication is done:

			0	1	0	1	0
x			1	1	0	0	1
-				1	0	1	0
+			1	0	1	0	
+		0	0	0	0		
-	1	0	1	0			

The sub are transformed into sum of the opposite, so the complement plus 1. The number of digits of the result is the sum of digits of the two numbers.

						0	1	0	1	0
x						1	1	0	0	1
+	1	1	1	1	1	1	0	1	1	0
+						1	0	1	0	
+					0	0	0	0		
+	1	1	1	0	1	1	0	0	0	0
$\cancel{1}$	1	1	1	0	1	1	1	0	1	0

E.2.2 Sum with redundant number system

We can have redundant number system where the number of digit allowed for a particular radix is higher than the radix itself.
To sum two number we must:

1. Sum the numbers digit by digit, creating a result that has a digit set doubled than the initial two numbers.

2. Define an intermediate sum and carry that sum up to the initial digit set, generating the carry using the radix.

3. Sum the intermediate sum with the carry.

Example:
We have a number system in radix 10 and digit set $[0, 11]$. The two number we want to sum are:

		11	10	7	11	3	8	digit $\in [0, 11]$
+		7	2	9	10	9	8	digit $\in [0, 11]$
		18	12	16	21	12	16	digit $\in [0, 22]$
		8	2	6	1	2	6	digit $\in [0, 9]$
	1	1	1	2	1	1		digit $\in [0, 2]$
	1	9	3	8	2	3	6	digit $\in [0, 11]$

E.2.3 Simmetric number representation

Usaually redundant number systems with radix r have a digit setin the form $[\overline{a}, a]$ where $\left\lceil \frac{r-1}{2} \right\rceil \leq a \leq r - 1$.
We can calculate the redundantancy of the digit set by dividing the number of numbers that have more than one representation by the total number in the range of digits for a defined number of digits n.
This digit sets permit to do the sum using an algorithm:

$$(x_{n-1}, \ldots, x_0) \pm (y_{n-1}, \ldots, y_0) = (s_{n-1}, \ldots, s_0)$$

1. Compute c_i with the rule:

$$c_i = \begin{cases} 1 & (x_i + y_1) \geq a \\ \overline{1} & (x_i + y_i) \leq \overline{a} \\ 0 & \overline{a} < (x_i + y_i) < a \end{cases}$$

2. Compute the interim sum u_i:

$$u_i = x_i + y_i - rc_i$$

3. Compute s_i:

$$s_i = u_i + c_{i-1}$$

Example:
We have a number system with $r = 10$ and $a = 6$.
With $n = 2$ the numbers in the range $\overline{66} \leq x \leq 66$ are 133, but we have $13^2 = 169$ number representation, so the redundantancy is:

$$(169 - 133)/133 = 27\%$$

We want to sum two numbers $X = 3645$ and $B = 1456$:

	3	6	4	5	X
+	1	4	5	6	Y
0	1	1	1		C
	4	0	$\overline{1}$	1	U
	5	1	0	1	S

E.2.4 Conversion between decimal and signed digit

Using the digit set as before $[\overline{a}, a]$ with radix $r = 10$, we can easily convert a number from decimal to this representation.

- Decimal to signed digit: Using the same formula as the sum but backwards, calculating the carry and interim sum.
- Signed digit to decimal: Simply subtract the negative digits.

Example:
We have the number 6849 in decimal and the conversion in signed digit with digit set $[\overline{6}, 6]$ is:

	6	8	4	9	X+Y
1	1	0	1		C
	$\overline{4}$	$\overline{2}$	4	$\overline{1}$	U
1	$\overline{3}$	$\overline{2}$	5	$\overline{1}$	S

The conversion backwards is simply:

	1	$\overline{3}$	$\overline{2}$	5	$\overline{1}$
	1	0	0	5	0
-	0	3	2	0	1
		6	8	4	9